

Exemplo - Módulo 3: Fine-Tuning via API (resolvido com Amplitude Seguros)

Disciplina: Processamento de Dados e Fine-Tuning de Modelos · Prof. Ahirton Lopes, Ph.D.

Missão Prática #03 - solução de referência

Caso escolhido: o dataset real da Amplitude Auto e Saúde Empresarial, o mesmo dos Módulos 3.2 a 3.5, duzentos exemplos, cento e vinte da Amplitude Auto, oitenta da Saúde Empresarial, treinado de verdade na Vertex AI, sob a marca Gemini Enterprise Agent Platform.

Passo 1 - Conversão pro formato Gemini

Cada exemplo do schema canônico (instrucao, entrada, saida, metadata) virou um objeto contents com dois turnos: role user, com a instrução mais o texto bruto da entrada; role model, com o JSON exato da saída esperada. Duzentos exemplos convertidos, salvos em m3-amplitude-auto-saude-200.jsonl, e subidos pro bucket gs://amplitude-seguros-demo-tuning.

Passo 2 - Hiperparâmetro validado, e um incidente real evitado da segunda vez

Config validada antes de qualquer chamada: epoch_count três (dentro da faixa um a vinte), learning_rate_multiplier cinco (dentro da faixa zero vírgula um a dez). Depois de criado, o job real, tuningJobs barra quatro um oito zero nove sete zero sete seis três seis cinco cinco oito três nove sete quatro quatro, foi consultado, e o hiperparâmetro aplicado bateu cem por cento com o pedido: epoch_count três, learning_rate_multiplier cinco, adapter_size ADAPTER_SIZE_FOUR.

Isso não é resultado óbvio: um teste real anterior, com epoch_count zero de propósito, mostrou que a Vertex AI aceita hiperparâmetro inválido sem rejeitar rápido, e substitui silenciosamente por um default. A comparação pedido versus aplicado é o que garante que isso não passou despercebido desta vez.

Passo 3 - Acompanhamento com backoff, testado sem rede

Função acompanharAteFinalizar implementada com backoff exponencial, cinco segundos de intervalo inicial, fator um vírgula cinco, teto de sessenta segundos. Testada com consultarFn e esperarFn falsas, simulando as sequências pending-running-running-succeeded e estado já terminal na primeira consulta, sem nenhuma chamada de rede real durante os testes. Contra o job real, a função reconheceu o estado succeeded já existente, sem precisar esperar nada, e devolveu o endpoint publicado.

Passo 4 - Ficha de versionamento completa

Hash SHA-256 real do dataset de duzentos exemplos: f6eb8f99c30e5ecc1c3ec5f293ba831bccfde59b7cc12070ad03169ab91b52ed. Ficha gerada, validada, com todos os seis campos obrigatórios presentes: job, modelo base gemini-2.5-flash, dataset e hash, hiperparâmetro, modelo ajustado projects/113512199474/locations/us-central1/models/391725756406824960@1, e endpoint projects/113512199474/locations/us-central1/endpoints/9091681879928602624.

Model card gerado em markdown, model-card-amplitude-auto-saude-m3-200.md, com identificação, linhagem, hiperparâmetro, estatística do dataset (duzentos exemplos, vinte e sete mil trezentos e cinquenta e três tokens cobráveis), e linha do tempo (criado às 2026-08-08T02:38:12Z, concluído às 2026-08-08T03:23:54Z, quarenta e cinco minutos e quarenta e dois segundos de treino real).

Execução do código

Ver model-versioning-tool.js e model_versioning_tool.py: catorze testes automatizados confirmam, em ambas as linguagens: hash determinístico e sensível a qualquer mudança de conteúdo, ficha completa gerada a partir de job bem formado, rejeição de job inválido, validação de completude, custo estimado calculado a partir da duração real do job, e geração de model card incluindo hash, identificadores reais e o custo real calculado certo.

Nota de curadoria

Reparem que o hash do dataset é idêntico entre a versão JavaScript e a versão Python, calculado sobre o mesmo arquivo, f6eb8f99c30e5ecc1c3ec5f293ba831bccfde59b7cc12070ad03169ab91b52ed nas duas. Isso não é coincidência de implementação, é a prova de que versionamento por conteúdo não depende de linguagem nem de biblioteca, só do dado em si, byte por byte. Ao fazer a própria atividade, testem isso de propósito: calculem o hash do mesmo arquivo duas vezes, em ferramentas diferentes se possível, e confirmem que bate.

Ahirton Lopes · Fine-Tuning Toolkit - UNIPDS: Processamento de Dados e Fine-Tuning de Modelos
Prof. Ahirton Lopes, Ph.D. - GDE AI, Microsoft MVP, Senior Manager